struct_draw

`struct_draw` is a high-precision Python library for generating CAD structural engineering drawings (DXF format). It converts structural design parameters and detailing calculations into production-ready CAD drawings featuring plan views, sectional elevations, reinforcement detailing, dimension lines, grid axes, and native AutoCAD `MULTILEADER` annotations.

Key Features & Capabilities

- **Production-Ready CAD Drawings**: Generates standard DXF files compatible with AutoCAD, BricsCAD, DraftSight, and LibreCAD.
- **Native AutoCAD `MULTILEADER` Entities**: Annotations for rebar, beams, and structural notes use CAD native `MULTILEADER` objects so text and leader arrows stay linked during manual CAD edits.
- **Dual Unit Support**: Full support for both **Imperial (US customary)** and **SI Metric** units, with explicit unit helper constructors (e.g., `FootingData.from_metric(...)`).
- **Comprehensive Structural Element Library**:
  - **Spread / Isolated Footings**: Plan and section views, sloped/stepped footing geometry, column dowels, hook bends, distribution mats, GL (Ground Level) / PL (Plinth Level) markers, and grid axes.
  - **Combined Footings**: Multi-column support, top & bottom longitudinal/transverse rebar mats, column starter dowels, dimensioning, and schedule tables.
  - **Staircases**: Waist-slab staircases featuring flight elevations, treads & risers, support beams, scissor/crank rebar at landing junctions, top & bottom extra support bars, distribution rebar, step nosing bars, binders, and plan view UP walklines.
  - **Beams & Columns**: Elevation and section reinforcement views, stirrups, legs, development length L_d, and schedule tables.
  - **Slabs**: One-way and two-way slab sectional reinforcement details.
- **Design Framework Adapters**: Seamlessly translates design results from structural design tools (e.g., `aci318`) into drawing data models.

Installation

```
pip install ezdxf
```

Or install `struct_draw` directly within your python environment:

```
pip install -e .
```

Quick Start Example: Spread Footing

Here is a complete example of generating a spread footing drawing (Plan + Section) in **SI Metric units**:

```
from pathlib import Path
import struct_draw as sd
from struct_draw.core.data import FootingData, Rebar, Stirrup

# 1. Define Footing Parameters in SI Metric Units (mm)
footing_si = FootingData.from_metric(
    name="F2",
    length_mm=1600.0,           # Footing length (1600 mm)
    width_mm=1600.0,            # Footing width (1600 mm)
    mid_thick_mm=400.0,         # Central footing thickness (400 mm)
    edge_thick_mm=200.0,        # Edge footing thickness (200 mm)
    col_width_mm=350.0,         # Column width (350 mm)
    col_depth_mm=350.0,         # Column depth (350 mm)
    col_height_mm=2100.0,       # Column height above footing (2100 mm)
    fd_depth_mm=1500.0,         # Foundation depth below Ground Level (1500 mm)
    pl_height_mm=600.0,         # Plinth Level height above Ground Level (600 mm)
    cover_mm=75.0,              # Concrete clear cover (75 mm)
    bar_dia_mm=12.0,            # Footing mat rebar diameter (12 mm)
    bar_spacing_mm=150.0,       # Rebar spacing center-to-center (150 mm c/c)
    ld_mm=600.0,                # Development length Ld (600 mm)
    column_bars=(
        Rebar(mark="C1", size="16mm dia", quantity=4, length_in=3000.0),
    ),
    column_stirrups=Stirrup(size="8mm dia", spacing_in=100.0, legs=2),
)

# 2. Render and save to DXF
output_path = Path("footing_drawing_si.dxf")
sd.save_dxf(footing_si, output_path)

print(f"Footing DXF successfully created at: {output_path.resolve()}")
```

Imperial Units Example

`struct_draw` also accepts Imperial parameters directly:

```
import struct_draw as sd
from struct_draw.core.data import FootingData, Rebar, Stirrup

footing_us = FootingData(
    name="F1",
    length_in=72.0,             # 6 ft length
    width_in=72.0,              # 6 ft width
    thickness_in=18.0,          # 1.5 ft thickness
    col_width_in=16.0,          # Column width
    col_depth_in=16.0,          # Column depth
    cover_in=3.0,               # Clear cover
    main_bar_dia_in=0.625,      # #5 rebar
```

```
    main_bar_spacing_in=6.0,    # 6" c/c
    column_bars=(Rebar(mark="C1", size="#6", quantity=4),),
    column_stirrups=Stirrup(size="#3", spacing_in=4.0, legs=2),
)

sd.save_dxf(footing_us, "footing_drawing_us.dxf")
```

Project Structure & Architecture

`struct_draw` follows a clean modular design that separates data models, CAD rendering primitives, structural element generators, CAD styling, and export backends:

```
struct_draw/
├── __init__.py              # Top-level exports and high-level drawing/saving
API functions
├── version.py               # Package version definition
├── adapters/                # Design code adapters (e.g. ACI 318 calculation
integration)
│   ├── __init__.py
│   └── aci318.py            # Converts ACI 318 design code objects into
struct_draw Data models
├── backends/                # DXF output generation backend (ezdxf integration)
│   └── __init__.py          # ezdxf document creation, layer setup, and DXF
exporter logic
├── core/                    # Core CAD rendering primitives, geometry engines, &
data structures
│   ├── __init__.py
│   ├── data.py              # Strongly-typed dataclasses for structural
parameters & rebar specs
│   ├── primitives.py        # Basic CAD entities (lines, polylines, circles,
rectangles, text, leaders)
│   ├── composite.py         # High-level CAD details (rebar hooks, dimensions,
grid bubbles, section callouts)
│   ├── document.py          # Document wrapper for layer, block, and viewport
management
│   ├── geometry.py          # Geometric transforms, vectors, and bounding box
calculations
│   ├── sheet.py             # Sheet border layouts, title blocks, and drawing
scale handling
│   └── viewport.py          # CAD viewport configuration and layout scaling
├── elements/                # Structural member drawing engines (Generates
complete detailing drawings)
│   ├── __init__.py
│   ├── footing.py           # Spread / isolated footing detailing (Plan &
Section views)
│   ├── combined_footing.py  # Combined multi-column footing detailing & schedule
tables
│   ├── staircase.py         # Waist-slab staircase elevation, flight details,
treads & risers
│   ├── beam.py              # Reinforced concrete beam elevation, section cuts,
```

```
& stirrup zones
│   ├── column.py            # Column elevation, cross-sections, & tie
arrangements
│   └── slab.py              # One-way and two-way slab sectional reinforcement
details
├── style/                   # CAD standards, layer colors, line types, & text
styles
│   ├── __init__.py
│   ├── blocks.py            # Standard CAD blocks (north arrow, section arrows,
level symbols)
│   ├── dimension_styles.py  # Dimension line arrowheads, tick marks, and
formatting
│   ├── layers.py            # Standardized CAD layer definitions and color
mappings (ACI/AIA standards)
│   ├── linetypes.py         # Custom CAD linetype definitions (hidden,
centerline, phantom)
│   └── text_styles.py       # Text font styles and character heights
└── utils/                   # Helper utilities and error handling
    ├── __init__.py
    ├── constants.py         # Physical constants, conversion factors (mm to
inches), and defaults
    ├── exceptions.py        # Custom exception hierarchy (DrawError,
ValidationError)
    └── geometry.py          # 2D vector mathematics and intersection helpers
```

Generated DXF drawings organize entities onto standard CAD layers:

Layer Name	Description
`CONC_OUTLINE` / `Slab` / `Beam`	Concrete outlines and member boundaries
`Rebar` / `REBAR`	Main longitudinal and flexural reinforcement
`Rebar_hatch`	Transverse / distribution rebar dots
`Stirrup`	Column ties, beam stirrups, and nosing binders
`Dimension`	Linear and aligned dimension lines
`CENTERLINE`	Column and grid centerline axes
`SECTION`	Section cut lines and callout symbols
`Text`	Annotations, titles, and multileader text

License

MIT License. See `LICENSE` for details.